Capstone Project Requirements Document

Full Stack Application Development using ASP.NET Core Web API and Angular

Abstract

This capstone project is to help participants apply the concepts learned during .NET and Angular training by developing a real-world full-stack web application.

Venkat Shiva Reddy
https://www.linkedin.com/in/venkatshivareddy

1. Project Overview

Objective

The objective of this capstone project is to help participants apply the concepts learned during .NET and Angular training by developing a real-world full-stack web application.

Participants will work in a team environment following Agile Scrum practices and deliver a working application in two sprints:

- Sprint 1: Backend Development using ASP.NET Core Web API
- Sprint 2: Frontend Development using Angular

The project is designed specifically for fresher developers to strengthen:

- Backend API development skills
- Database design and Entity Framework Core usage
- RESTful service development
- Angular frontend development
- Team collaboration and Agile practices
- Source code management using Git
- Testing and debugging
- Application architecture and clean coding practices

---

2. Team Details

Item	Details
Total Participants	5 to 6
Project Duration	2 Weeks
Total Sprints	2
Sprint Duration	1 Week Each
Methodology	Agile Scrum
Technologies	ASP.NET Core Web API, Angular, SQL Server

---

3. Suggested Project Domain

Project Title

Smart Task & Employee Management System

The application helps organizations manage:

- Employees
- Projects
- Tasks
- Task Assignments
- Status Tracking
- Dashboard Reporting

The system should allow managers and employees to manage project tasks efficiently.

---

4. Functional Modules

Module 1: User Management

Features

- User Registration
- User Login
- Role Management
  - Admin
  - Manager
  - Employee
- JWT Authentication
- Password Validation
- User Profile Management

Backend Requirements

- REST APIs for authentication and user operations
- JWT Token generation
- Password hashing
- Role-based authorization

Frontend Requirements

- Login page
- Registration page
- Profile page
- Navigation menu based on role

Module 2: Employee Management

Features

- Add Employee
- Update Employee
- Delete Employee
- Search Employees
- View Employee Details
- Employee List with Pagination

Employee Fields

- Employee ID
- Name
- Email
- Phone Number
- Department
- Designation
- Joining Date

Backend Requirements

- CRUD REST APIs
- Input validation
- Exception handling
- Pagination support

Frontend Requirements

- Employee list page
- Add/Edit employee form
- Search and filter functionality
- Validation messages

Module 3: Project Management

Features

- Create Project
- Assign Project Manager
- Update Project Details
- Delete Project
- View Project Summary

Project Fields

- Project ID
- Project Name
- Client Name
- Start Date
- End Date
- Budget
- Status

Backend Requirements

- CRUD APIs
- Relationship mapping using Entity Framework Core
- DTO usage

Frontend Requirements

- Project dashboard
- Project forms
- Project detail page

Module 4: Task Management

Features

- Create Task
- Assign Task to Employee
- Update Task Status
- Track Progress
- Task Priority Management

Task Status

- New
- In Progress

- Completed
- Blocked

Task Priority

- Low
- Medium
- High

Backend Requirements

- Task APIs
- Task assignment logic
- Filtering APIs

Frontend Requirements

- Task board/grid
- Task assignment form
- Status update UI

Module 5: Dashboard & Reports

Features

- Total Employees
- Total Projects
- Total Tasks
- Completed Tasks
- Pending Tasks
- Project Status Summary

Backend Requirements

- Aggregated reporting APIs
- LINQ queries for statistics

Frontend Requirements

- Dashboard page
- Charts and summary cards
- Responsive design

---

5. Non-Functional Requirements

Performance

- APIs should respond within reasonable time
- Use asynchronous programming where applicable

Security

- JWT Authentication mandatory
- Passwords must be hashed
- Sensitive APIs must be protected

Usability

- Simple and user-friendly UI
- Responsive Angular design

Maintainability

- Follow clean architecture principles
- Use layered architecture
- Use meaningful naming conventions

Scalability

- Proper project structure
- Separation of concerns

---

6. Recommended Architecture

Backend Architecture

Use layered architecture:

- Controllers
- Services
- Repositories
- Data Layer
- Models/Entities
- DTOs

Frontend Architecture

Use Angular best practices:

- Components
- Services
- Routing
- Shared Modules
- Guards
- Reactive Forms

---

7. Technology Stack

Backend

- ASP.NET Core Web API (.NET 8 preferred)
- Entity Framework Core
- SQL Server
- Swagger/OpenAPI
- JWT Authentication

Frontend

- Angular
- TypeScript
- Angular Material or Bootstrap
- RxJS
- Reactive Forms

Tools

- Visual Studio / VS Code
- SQL Server Management Studio
- Postman
- Git & GitHub

---

8. Database Design Requirements

Mandatory Tables

Users

- UserId
- Name
- Email
- PasswordHash
- Role

Employees

- EmployeeId
- Name
- Department
- Designation
- Email

Projects

- ProjectId
- Name
- ClientName
- StartDate
- EndDate

Tasks

- TaskId
- Title
- Description
- Status
- Priority
- EmployeeId
- ProjectId

---

9. REST API Requirements

API Standards

Participants must follow RESTful principles:

Operation	HTTP Method
Get Data	GET
Create Data	POST
Update Data	PUT
Delete Data	DELETE

API Best Practices

- Use proper status codes
- Use DTOs
- Validate request models
- Handle exceptions globally
- Use meaningful routes

Example

GET /api/employees
POST /api/employees
PUT /api/employees/{id}
DELETE /api/employees/{id}

---

10. Angular Frontend Requirements

Mandatory Angular Features

Routing

- Lazy loading preferred
- Route guards for authentication

Forms

- Reactive Forms mandatory
- Form validations required

HTTP Communication

- Angular HttpClient
- API integration
- Error handling

UI Requirements

- Responsive layout
- Navigation menu
- Loading indicators
- Validation messages

---

11. Sprint Planning

Sprint 1 — Backend Development

Duration

5 Working Days

Sprint Goal

Develop complete RESTful backend services with database integration.

Sprint Tasks

Day 1

- Requirement understanding
- Database design
- Create solution structure
- Configure EF Core

Day 2

- Create entities and DbContext
- Implement repositories
- Create DTOs

Day 3

- Develop CRUD APIs
- Implement validations
- Add Swagger

Day 4

- Implement JWT Authentication
- Implement role-based authorization
- Exception handling

Day 5

- API testing using Postman
- Bug fixing
- Sprint review

Sprint 1 Deliverables

- Working APIs
- SQL scripts
- Swagger documentation
- Postman collection
- Git repository updated

Sprint 2 — Angular Frontend Development

Duration

5 Working Days

Sprint Goal

Develop Angular frontend and integrate with backend APIs.

Sprint Tasks

Day 1

- Angular project setup
- Create modules and routing
- Setup UI framework

Day 2

- Develop login and authentication
- Configure route guards
- Create shared services

Day 3

- Develop employee and project modules
- API integration

Day 4

- Develop task management screens
- Dashboard implementation
- Form validations

Day 5

- End-to-end testing
- Bug fixing
- Final demo preparation

Sprint 2 Deliverables

- Working Angular application
- API integration completed
- Responsive UI
- Git repository updated
- Final project demo

---

12. Team Responsibilities

Suggested Team Distribution

Participant	Responsibility
Member 1	Authentication & Security
Member 2	Employee Module
Member 3	Project Module
Member 4	Task Module
Member 5	Dashboard & Reports
Member 6	Frontend Integration & Testing

Note:
All participants should contribute to both backend and frontend implementation.

---

13. Coding Standards

Backend Standards

- Use meaningful class and method names
- Follow SOLID principles
- Use async/await where applicable
- Separate business logic from controllers
- Use dependency injection

Frontend Standards

- Use reusable Angular components
- Use services for API communication
- Avoid hardcoded values
- Use proper folder structure

---

14. Git & Collaboration Guidelines

Mandatory Git Practices

- Use GitHub repository
- Commit code daily
- Use meaningful commit messages
- Create feature branches
- Raise pull requests for review

Example Commit Messages

- Added employee CRUD APIs
- Implemented JWT authentication
- Created Angular login component

---

15. Testing Requirements

Backend Testing

- Test all APIs using Postman
- Verify status codes
- Test invalid scenarios

Frontend Testing

- Validate forms
- Verify navigation
- Test API integration
- Test responsive design

---

16. Evaluation Criteria

Criteria	Weightage
Backend Functionality	25%
Angular Frontend	25%
API Integration	15%
Database Design	10%
Code Quality	10%
Team Collaboration	5%
Presentation & Demo	10%

---

17. Final Deliverables

Participants must submit:

Source Code

- Backend project
- Angular project

Database

- SQL scripts
- Database schema

Documentation

- API documentation
- Setup instructions
- Architecture diagram

Demo

- End-to-end application demo
- Explanation of architecture and implementation

---

18. Optional Advanced Features

Participants can implement additional features for bonus evaluation:

- Email notifications
- File upload
- Export reports to Excel/PDF
- Dark mode UI
- Audit logging
- Real-time notifications using SignalR
- Unit testing
- Docker containerization

---

19. Expected Learning Outcomes

By completing this capstone project, participants should be able to:

- Develop RESTful APIs using ASP.NET Core
- Implement authentication and authorization
- Work with Entity Framework Core
- Design relational databases
- Build Angular frontend applications
- Integrate frontend with backend APIs
- Follow Agile Scrum practices
- Use Git for collaboration
- Apply clean coding practices
- Deliver a complete full-stack application

---

20. Success Criteria

The project will be considered successful if:

- All core modules are implemented
- Backend APIs are functional
- Angular frontend is integrated successfully

- Application works end-to-end
- Code follows clean architecture principles
- Team demonstrates collaboration and understanding

---

21. Submission Timeline

Milestone	Timeline
Sprint 1 Review	End of Week 1
Sprint 2 Review	End of Week 2
Final Demo	Last Day
Final Submission	Last Day

---

22. Guidance for Participants

Recommended Daily Activities

- Attend daily stand-up meetings
- Update task status regularly
- Commit code frequently
- Ask questions early when blocked
- Collaborate with teammates
- Test features before pushing code

Important Notes

- Focus on learning over perfection
- Maintain clean and readable code
- Use proper naming conventions
- Avoid copying code without understanding
- Ensure every member participates actively

---

23. Recommended Folder Structure

Backend Structure

- Controllers
- Services
- Repositories
- DTOs

- Models
- Data
- Middleware
- Helpers

Angular Structure

- components
- services
- models
- guards
- interceptors
- shared
- layouts

---

24. Suggested Agile Ceremonies

Daily Stand-Up

Duration: 15 Minutes

Each participant should answer:

- What did I complete yesterday?
- What will I work on today?
- Any blockers?

Sprint Review

- Demonstrate completed features
- Gather feedback
- Discuss improvements

Sprint Retrospective

- What went well?
- What can be improved?
- Action items for next sprint

---

25. Conclusion

This capstone project is intended to simulate a real-world software development experience for fresher developers. Participants are expected to apply their training knowledge practically while learning collaboration, problem-solving, and professional development practices.

The focus should be on:

- Learning
- Team collaboration
- Practical implementation
- Code quality
- End-to-end application development

Best wishes for successful project implementation.